Documentation

Personal Accident Insurance Management System

Submitted By:
Abhishek Allaboina

Capstone Project — The Hartford Training Program
GitHub: https://github.com/abshek7/ABHISHEK

Table of Contents

Table of Contents....2
1. Abstract....4
2. Problem Statement....5
3. Project Objectives....6
4. Technology Stack and Core Engines....7
4.1. Server-Side Technologies (Backend)....7
4.2. Client-Side Technologies (Frontend)....8
5. System Architecture....9
5.1. Backend Architecture Layers....9
5.2. Frontend Architecture....9
5.3. Interaction Flow Example (Raising a Claim)....10
6. Project Structure....11
6.1. Backend Structure (CapstoneBackend)....11
6.2. Frontend Structure (CapstoneFrontend)....11
7. Advanced Business Logic Engines....13
6.1. Underwriting Engine (Risk Assessment)....13
6.2. Least Workload Assignment Algorithm....13
6.3. Coverage Category System....14
8. Detailed Module Explanations....15
7.1. Admin Module....15
7.2. Customer Module....15
7.3. Agent Module....16
7.4. Claims Officer Module....16
9. Complete Business Workflow....17
8.1. Policy Lifecycle: Registration to Activation....17
8.2. Claim Processing Lifecycle: Incident to Closure....17
10. Entity-Relationship (ER) Diagram and Database Design....19
9.1. Entity: User....19
9.2. Entity: PolicyType....20
9.3. Entity: PolicyCoverage....20
9.4. Entity: PolicyRequest....21

9.5. Entity: Policy 22
9.6. Entity: InsuranceClaim 24
9.7. Entity: ClaimReview 24
9.8. Entity: Settlement 25
9.9. Entity: Payment 26
9.10. Entity: Nominee 26
9.11. Entity Relationship Summary 27
11. Database Enumerations 28
12. API Documentation Overview 29
11.1. Authentication Endpoints (/api/Auth) 29
11.2. Customer Operations (/api/Customer) 29
11.3. Agent Operations (/api/Agent) 29
11.4. Claims Officer Operations (/api/ClaimsOfficer) 30
11.5. Administrative Operations (/api/Admin) 30
11.6. Support Services 30
13. Frontend Component Architecture 32
12.1. Core Services and State Management 32
12.2. Public Components 32
12.3. Customer Dashboard Components 33
12.4. Agent Dashboard Components 33
12.5. Claims Officer Dashboard Components 33
12.6. Admin Console Components 34
12.7. Shared and Layout Components 34
14. Key Features and Technical Highlights 35
13.1. Key Features 35
13.2. Development Challenges and Solutions 35
15. Future Enhancements 37
16. Conclusion 38

1. Abstract

The Personal Accident Insurance Management System is a capstone project developed as part of The Hartford training program. It is an advanced, centralized digital platform engineered to modernize and streamline the traditional, multi-faceted processes associated with personal accident insurance. Historically, the insurance sector has relied heavily on manual data entry, physical document handling, and localized data storage, leading to systemic inefficiencies, delayed claim processing, and frequent miscommunication among stakeholders.

The fundamental purpose of this system is to digitize and automate the entire insurance lifecycle from initial customer onboarding, dynamic premium underwriting, and policy issuance to the complex orchestration of claim settlements involving multiple discrete roles. By implementing an end-to-end digital infrastructure, the system significantly reduces turnaround times, eliminates human error in premium calculations, and ensures transparent, traceable communication across all stakeholders.

The expected benefits encompass enhanced customer satisfaction through self-service portals, fortified data security through robust role-based authorization models, accelerated operational throughput, and comprehensive fraud prevention via mandatory document verification and multi-stage claim processing. Ultimately, this system provides a highly scalable, secure, and user-centric application empowering policyholders and insurance personnel with specialized, role-driven dashboards and automated workflows.

2. Problem Statement

The traditional personal accident insurance industry suffers from several critical operational pain points that undermine efficiency, profitability, and customer trust:

- **Excessive Manual Processing and Paperwork:** A high reliance on physical documents for policy issuance and claim evidence collection leads to substantial administrative overhead, archiving costs, a high risk of document loss, and slow information retrieval times.
- **Delayed Claim Settlements:** The absence of integrated digital workflows means claims must be manually transitioned between customers, claims officers, and administrators, introducing severe bottlenecks and weeks-long resolution delays.
- **Lack of Transparency and Traceability:** Customers have no real-time visibility into the status of their policies or claims. Internally, tracking claim status or responsible officers becomes a managerial challenge.
- **Complex Underwriting and Premium Calculation:** Manual calculations for risk assessment based on personal habits, occupation, age, and medical history are inherently error-prone, leading to financial discrepancies and inconsistent pricing.
- **Approval Bottlenecks and Security Risks:** Without automated, role-based approval queues, policies and claims stagnate in backlogs. Physical files also lack granular access control, exposing sensitive data to unauthorized personnel.
- **Workload Imbalance:** Manual assignment of claims and policy requests to officers and agents leads to uneven workloads, reducing overall efficiency and increasing processing delays.

3. Project Objectives

The primary objectives of this system are to engineer a robust, enterprise-grade software ecosystem capable of automating and managing the full spectrum of personal accident insurance activities:

1. **Automate Policy Operations:** Digitize the purchasing, tracking, modification, and renewal of insurance policies, including a sophisticated underwriting engine that evaluates risk factors such as age, occupation, personal habits, and medical history to calculate personalized premiums.
2. **Centralize Customer Records:** Establish a secure, consolidated, and easily queryable repository for all customer biographical details, policy records, claims history, nominees, and historical transaction logs.
3. **Streamline Claim Processing:** Architect a transparent, multi-stage pipeline for raising claims, securely uploading digital evidence, assigning claims officers based on workload, and securing final financial approval with automated settlement processing.
4. **Implement Strict Role-Based Access Control (RBAC):** Ensure data security and clear segregation of duties across four distinct organizational roles: Admin, Agent, Claims Officer, and Customer.
5. **Enhance Workflow Efficiency:** Utilize automated status transitions, customized dashboards, and intelligent workload balancing to ensure no policy or claim remains stalled in the pipeline.
6. **Integrate Intelligent Features:** Incorporate an AI-powered chatbot built on Microsoft Semantic Kernel for customer assistance, analytics dashboards for business insights, and a least-workload algorithm for intelligent task assignment.

4. Technology Stack and Core Engines

The system is designed as a modern, full-stack web application utilizing a decoupled client-server architecture. It integrates several specialized engines and libraries to handle distinct functional requirements efficiently.

4.1. Server-Side Technologies (Backend)

Layer / Category	Technology	Purpose and Details
Framework	ASP.NET Core 8.0 Web API	Clean Architecture (Domain-Driven Design) with API, Application, Domain, and Infrastructure layers
ORM Engine	Entity Framework (EF) Core	Intuitive data access, schema migrations, and LINQ-based querying with SQL Server
Auth Engine	JSON Web Tokens (JWT)	Stateless, secure API communication with role-claim authorization (RBAC)
Database	Microsoft SQL Server	Structured, ACID-compliant relational data storage
AI Chat Bot	Microsoft Semantic Kernel	AI orchestration framework for building the intelligent chatbot that assists users with policy queries and claim guidance using natural language
Object Mapping	Auto Mapper	Efficient, convention-based mapping between domain entities and DTOs, reducing boilerplate in service and controller layers
API Docs	Swagger / Open API	Interactive endpoint visualization, testing, and contract definitions
Testing	xUnit + Moq	Unit tests for controllers, services, and repositories using xUnit as the test framework and Moq for dependency mocking

4.2. Client-Side Technologies (Frontend)

Layer / Category	Technology	Purpose and Details
Framework	Angular 21 (TypeScript)	Modular, reactive Single Page Application (SPA) with Standalone Components
UI Component Library	NG-ZORRO (Ant Design)	Premium, professional UI components providing a polished, consistent interface
State Management	Angular Signals and RxJS	Angular Signals for fine-grained reactive component state; RxJS for managing asynchronous data streams, HTTP responses, polling, and complex event-driven logic
Responsive Design	Ant Design Grid System	Fluid, mobile-first layout with responsive breakpoints across all screen sizes
Styling	Custom CSS	Modern gradients, shadows, and micro-interactions for a premium user experience
Form Handling	Angular Reactive Forms	Comprehensive client-side form validation and multi-step wizard support
Icons	Lucide Icons	Modern, lightweight icon system used throughout the interface for consistent visual language
Onboarding	Driver.js	Interactive step-by-step onboarding tour for new users, guiding them through key features on first login

5. System Architecture

The project relies on a decoupled, multi-tiered RESTful architecture following Clean Architecture principles, logically divided into four distinct layers to promote separation of concerns and testability.

5.1. Backend Architecture Layers

- **API Layer:** Exposes RESTful endpoints and handles request/response formatting, JWT authentication, and global exception handling. Maps HTTP verbs to endpoint controllers and enforces [Authorize] attributes to verify JWT validity and RBAC permissions before routing to the service layer.
- **Application Layer:** Contains business logic interfaces, DTOs (Data Transfer Objects), service implementations, and the underwriting engine. This is where domain rules are enforced including premium calculation, risk scoring, claim eligibility verification, and workload balancing.
- **Domain Layer:** Defines core entities (User, Policy Request, Policy, Insurance Claim, etc.), enumerations (User Role, PolicyStatus, ClaimStatus), and fundamental domain logic that is independent of any infrastructure concerns.
- **Infrastructure Layer:** Implements data access using Entity Framework Core with SQL Server, file storage for uploaded documents, and the Microsoft Semantic Kernel AI ChatBot integration.

5.2. Frontend Architecture

- **Role-Based Dashboard Rendering:** The Angular frontend dynamically configures navigation menus, accessible routes, and visible components based on the decoded JWT payload containing the user role.
- **Reactive Data Management:** Angular Signals manage component-level reactive state for instant UI updates, while RxJS Observables handle HTTP communication, stream transformations, and complex asynchronous event chains such as real-time claim status polling using interval and switchMap operators.
- **Modular Component Design:** Components are organized by role and feature, each communicating with backend services via Angular's HTTP client using strongly typed DTOs.

5.3. Interaction Flow Example (Raising a Claim)

```
Frontend (Angular) ->
POST /api/Customer/claim + JWT ->

API Layer -> Token & Role Validation ->

Claim Service (Verify Policy Active, Verify Coverage) ->

Repository (Insert Claim) -> Database

-> Success Response

-> UI Update
```

6. Project Structure

The repository at https://github.com/abshek7/ABHISHEK contains three top-level folders: CapstoneBackend, CapstoneFrontend, and Deliverables. The backend follows Clean Architecture with four distinct projects and the frontend follows Angular feature-module conventions.

6.1. Backend Structure (CapstoneBackend)

Project / Folder	Contents
CapStone.API	Controllers (Admin, Agent, Auth, Customer, Claims, ChatBot), Middleware for global exception handling, Program.cs with dependency injection configuration
CapStone.Application	DTOs for all request and response models, Service interfaces defining business operation contracts, Repository interfaces and Unit of Work pattern, AutoMapper profiles for entity-to-DTO mapping, Custom application exceptions
CapStone.Domain	All database entities (User, Policy, PolicyRequest, InsuranceClaim, Payment, Nominee, Settlement, ClaimReview, PolicyType, PolicyCoverage), Enumerations (UserRole, PolicyStatus, ClaimStatus, RequestStatus, PaymentStatus, CoverageCategory)
CapStone.Infrastructure	DbContext and seed data, Repository implementations, Business service implementations including the Underwriting Engine and Semantic Kernel ChatBot service, Entity Framework Core database migrations
CapStone.Test	Unit tests for controllers, services, and repositories using xUnit as the test framework and Moq for mocking dependencies

6.2. Frontend Structure (CapstoneFrontend)

Folder	Contents
src/app/components/	Feature-specific components organized by role: login, admin-dashboard, agent-dashboard, customer-dashboard, public-page (landing page and chatbot)
src/app/services/	API integration and business logic services: auth (JWT and role management), policy (insurance operations), claims (claims processing)
src/app/models/	TypeScript interfaces and DTOs for strong typing across all API communications

Folder	Contents
src/app/guards/	Route protection guards (RoleGuard) preventing unauthorized access to role-specific dashboards
src/app/interceptors/	HTTP interceptors for automatic JWT token injection into all outbound API requests

7. Advanced Business Logic Engines

The backend is powered by several sophisticated engines that automate complex insurance underwriting and claim processing workflows dynamically.

6.1. Underwriting Engine (Risk Assessment)

The Underwriting Engine evaluates personal risk factors for each policy application using a multiplier-based risk model. Multiple independent evaluators assess different risk dimensions and their results are multiplied together to produce a final composite risk score that determines both eligibility and premium.

Risk Factor	Evaluator Class	Logic and Rule
Age	AgeEvaluator	Young or elderly applicants receive higher multipliers due to increased statistical accident risk.
Occupation	OccupationEvaluator	High-risk occupations such as construction, mining, or heavy machinery operation return elevated multipliers.
Personal Habits	HabitsEvaluator	Smoking, alcohol consumption, and engagement in dangerous recreational activities increase the risk multiplier.
Medical History	MedicalHistoryEvaluator	Pre-existing conditions, chronic illness, or prior disability claims significantly elevate the multiplier.
Final Risk Score	Composite Calculation	FinalRiskScore = Multiplier1 x Multiplier2 x Multiplier3 x BaseMultiplier (product of all evaluator results).
Eligibility Threshold	Eligibility Check	If FinalRiskScore > 5, the policy request is automatically flagged and rejected. The customer is notified with the reason.
Premium Calculation	PremiumCalculator	Premium = BasePremium x FinalRiskScore. Eligible applications (score <= 5) proceed to policy creation and payment.

6.2. Least Workload Assignment Algorithm

When a new policy request or insurance claim is received, the system automatically assigns it to the agent or claims officer carrying the lowest current active workload, ensuring equitable distribution and optimal processing throughput.

Step	Description
1. Query Active Assignments	Fetch the count of currently active (pending or in-review) tasks for each eligible agent or claims officer.
2. Sort by Workload	Sort the staff list in ascending order by active task count.
3. Assign to Least Busy	Assign the incoming request to the staff member at the top of the sorted list (minimum workload).
4. Persist and Confirm	Persist the assignment in the database and send an in-app notification to the assigned staff member.

6.3. Coverage Category System

Each policy type supports multiple configurable coverage categories that define the scope and financial limits of insurance protection available to the policyholder:

Coverage Category	Description	Payout Basis
Accidental Death	Full coverage payout to the nominated beneficiary in the event of death resulting directly from an accident.	Lump sum equal to full coverage amount
Permanent Total Disability	Payout for permanent and total loss of functional capacity due to accidental injury.	Lump sum based on configured percentage of coverage
Permanent Partial Disability	Proportional payout based on the assessed degree of permanent partial disability sustained.	Percentage of coverage proportional to disability degree
Temporary Total Disability	Weekly compensation during temporary inability to work, calculated based on recovery duration.	Weekly compensation for the assessed recovery period

8. Detailed Module Explanations

The system employs granular Role-Based Access Control to partition functionality across four core user groups, ensuring that users interact only with data and operations pertinent to their organizational role.

7.1. Admin Module

The Admin is the super-user responsible for system configuration, user management, and macro-level oversight of the entire platform.

- Agent and Officer Registration: Creates and provisions new Agent and Claims Officer accounts with appropriate role assignments and access credentials.
- Workload-Based Assignment: Uses the Least Workload Algorithm to automatically assign incoming policy requests to agents and new claims to claims officers.
- Analytics Dashboard: Access to aggregated data visualizations detailing system health, total revenue collected, claim resolution ratios, and policy distribution metrics.
- Revenue Reports: Detailed financial breakdowns of premiums collected, claims settled, agent commissions earned, and overall business performance.
- Policy and User Management: Global oversight of all registered users, active policies, pending requests, and system-wide claims.

7.2. Customer Module

The Customer is the primary end-user seeking personal accident insurance coverage and the initiator of the claims process.

- Policy Marketplace: Browse available insurance products with detailed coverage descriptions, premium estimates, and plan comparisons.
- Policy Application: Multi-step form submission including personal information, medical history, lifestyle habits, and supporting document upload for underwriting assessment.
- Premium Payment: Record premium payments to activate policies following successful underwriting approval.
- Claim Filing: Guided wizard for reporting accidents, selecting the coverage category, specifying the claim amount, and uploading supporting documents.
- Nominee Management: Add or update primary and secondary nominees for each active policy with relationship and contact information.
- Notifications and History: View policy status updates, claim progress, payment history, and renewal reminders.

7.3. Agent Module

The Agent is the intermediary representative who assists customers with policy acquisition and performs underwriting assessments on assigned policy requests.

- Assigned Request Queue: A prioritized queue displaying policy requests assigned to this agent for risk evaluation.
- Risk Assessment Interface: Input evaluation factors for each applicant and trigger the Underwriting Engine to compute the risk score, determine eligibility, and calculate the recommended premium.
- Direct Policy Initiation: Ability to initiate a policy application on behalf of a customer, streamlining the onboarding process.
- Commission Summary: Track total commissions earned from successfully activated policies.

7.4. Claims Officer Module

The Claims Officer is the final decision-maker responsible for reviewing, assessing, approving or rejecting insurance claims, and processing settlements.

- Review Queue: Streamlined interface for managing the queue of assigned insurance claims with status indicators and claim amounts.
- Decision Support: View complete incident details, uploaded documents, policy coverage limits, and full claim history to make informed decisions.
- Claim Review Submission: Record detailed assessment findings including disability percentage, estimated recovery weeks, and fraud risk indicators.
- Settlement Processing: Define the final approved payout amount and initiate settlement processing for approved claims.

9. Complete Business Workflow

8.1. Policy Lifecycle: Registration to Activation

7. **Customer Onboarding:** A new user self-registers creating a User profile with Customer privileges, providing essential biographical details and account credentials.
8. **Policy Marketplace Browsing:** The customer browses available policy types and their coverage categories, reviewing base premiums and coverage amounts.
9. **Policy Application Submission:** The customer, or an Agent on their behalf, submits a policy request including medical history, personal habits, occupation details, and nominee information.
10. **Underwriting Assessment:** An assigned Agent performs the underwriting assessment. The system evaluates multiple risk factors to compute a composite risk score and calculate the personalized premium.
11. **Eligibility Decision:** If the final risk score exceeds 5, the policy request is automatically rejected and the customer is notified with the reason. Eligible requests proceed to the payment stage.
12. **Premium Payment and Activation:** Upon successful premium payment, the system generates an active Policy record with defined coverage amounts and start and end dates, and sends a confirmation to the customer.
13. **Policy Lifecycle Management:** Active policies generate renewal reminders as the expiry date approaches. Upon expiry, customers can submit renewal requests to continue coverage.

8.2. Claim Processing Lifecycle: Incident to Closure

14. **Claim Initiation:** The customer raises a claim against an Active policy, providing incident details, selecting the coverage category, specifying the claim amount, and uploading supporting documents. Claim status is set to Submitted.
15. **Administrative Assignment:** Admin assigns the claim to a Claims Officer using the Least Workload Algorithm, ensuring equitable distribution across the officer team.
16. **Document Verification:** The Claims Officer reviews uploaded evidence and incident details. If documents are insufficient, additional documentation is requested from the customer.
17. **Claim Investigation:** The Officer investigates the claim, assessing the disability percentage for disability claims, recovery timeline, and potential fraud indicators.
18. **Decision and Settlement:** For approved claims, the Officer enters the final approved amount and initiates settlement processing. The claim status transitions to Settled and the

customer is notified. For rejected claims, the officer records a rejection reason and the customer is notified accordingly.

10. Entity-Relationship (ER) Diagram and Database Design

The database design is based on a fully normalized relational schema that models all entities, their attributes, and the relationships between them. The schema is designed to Third Normal Form (3NF) to ensure data integrity, eliminate redundancy, and accurately track all lifecycle state transitions. This section provides a detailed description of every entity, its fields, data types, constraints, and its relationships within the system.

9.1. Entity: User

The User entity is the central identity record for every participant in the system. A single User record serves all four roles differentiated by the Role field. This unified design avoids duplicate identity tables and simplifies authentication and authorization.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for each user record.
Name	string, NOT NULL	Full legal name of the user.
Email	string, UNIQUE, NOT NULL	Email address used for login. Must be unique across all users.
PasswordHash	string, NOT NULL	Bcrypt-hashed password. Plain-text passwords are never stored.
Role	Enum (UserRole), NOT NULL	Defines system access level: Admin, Agent, Customer, or ClaimsOfficer.
DateOfBirth	DateTime, nullable	Date of birth used in underwriting age risk evaluation.
Phone	string, nullable	Contact phone number for communication.
Address	string, nullable	Residential address of the customer.
Occupation	string, nullable	Used by the OccupationEvaluator in the underwriting risk engine.
PersonalHabits	string, nullable	Description of habits (e.g. smoking, alcohol) used in risk scoring.
IsActive	bool, NOT NULL	Soft-delete flag. Inactive users cannot log in but records are preserved.
CreatedAt	DateTime, NOT NULL	Timestamp of account creation.

Relationships: One User (Customer) has many PolicyRequests. One User (Agent) is assigned to many PolicyRequests and Policies. One User (ClaimsOfficer) is assigned to many InsuranceClaims and ClaimReviews. One User (Admin) creates PolicyTypes.

9.2. Entity: PolicyType

The PolicyType entity defines the master catalogue of insurance products offered by the company. Each PolicyType sets the baseline financial parameters and duration that apply to all policies issued under that product type.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the policy type.
Name	string, NOT NULL	Product name, for example Basic Accident Cover or Premium Shield Plan.
BasePremium	decimal, NOT NULL	The starting annual premium amount before risk adjustment by the underwriting engine.
BaseCoverageAmount	decimal, NOT NULL	The maximum total insurance payout available under this policy type.
DurationMonths	int, NOT NULL	Standard coverage duration in months for policies issued under this type.
CreatedBy	GUID (FK -> User), NOT NULL	References the Admin user who created this policy type.

Relationships: One PolicyType has many PolicyRequests. One PolicyType has many PolicyCoverages defining the coverage breakdown per category. Many Policies are issued under one PolicyType.

9.3. Entity: PolicyCoverage

The PolicyCoverage entity defines the individual coverage categories available within a specific PolicyType along with the financial parameters for each category. This allows flexible, configurable coverage compositions per insurance product.

Field	Type / Constraint	Description

Id	GUID (PK)	Unique identifier for the coverage record.
PolicyTypeId	GUID (FK -> PolicyType), NOT NULL	References the parent PolicyType this coverage category belongs to.
CoverageCategory	Enum (CoverageCategory), NOT NULL	Category: AccidentalDeath, PermanentTotalDisability, PermanentPartialDisability, or TemporaryTotalDisability.
PercentageOfCoverage	decimal, NOT NULL	The percentage of BaseCoverageAmount that applies to this category (e.g. 100% for death).
WeeklyCompensationPercentage	decimal, nullable	For TemporaryTotalDisability: the weekly payout as a percentage of base coverage.
MaxWeeks	int, nullable	Maximum number of weeks of weekly compensation payable for temporary disability claims.
Description	string, NOT NULL	Detailed human-readable description of what this coverage category includes and excludes.

Relationships: Many PolicyCoverages belong to one PolicyType. Coverage terms defined here are inherited at policy issuance time by the associated Policy through its linked PolicyType.

9.4. Entity: PolicyRequest

The PolicyRequest entity represents a customer's application for a new insurance policy. It captures all information required for underwriting assessment and records the full result including risk score, calculated premium, and eligibility determination before a formal Policy record is created.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the policy application.
CustomerId	GUID (FK -> User), NOT NULL	References the Customer submitting the application.
PolicyTypeId	GUID (FK -> PolicyType), NOT NULL	References the insurance product being applied for.

AssignedAgentId	GUID (FK -> User), nullable	References the Agent assigned via the Least Workload Algorithm.
RequestDate	DateTime, NOT NULL	Timestamp when the application was submitted by the customer.
Status	Enum (RequestStatus), NOT NULL	Application state: New, Assigned, UnderReview, Approved, Rejected, or Accepted.
MedicalHistory	string, nullable	Customer-provided description of medical history used in underwriting.
DocumentFilePath	string, nullable	Server file path to the uploaded supporting identity or medical documents.
TotalRiskScore	decimal, nullable	Composite risk score computed by the underwriting engine after assessment.
CalculatedPremium	decimal, nullable	Personalized annual premium = BasePremium x TotalRiskScore, populated after underwriting.
CoverageAmount	decimal, nullable	Final coverage amount approved for this policy, populated after underwriting.
IsEligible	bool, nullable	True if TotalRiskScore <= 5. False if score exceeds the threshold, triggering automatic rejection.
NomineeName	string, nullable	Name of the nominated beneficiary for death benefit claims.
NomineeRelation	string, nullable	Relationship of the nominee to the applicant (e.g. Spouse, Child).
IsPaid	bool, NOT NULL	Indicates whether the premium payment has been successfully completed.
PaymentDate	DateTime, nullable	Timestamp of confirmed payment, which triggers policy activation.

Relationships: Many PolicyRequests belong to one Customer (User). One PolicyRequest is assigned to one Agent (User). One PolicyRequest references one PolicyType. One approved and paid PolicyRequest results in exactly one Policy record.

9.5. Entity: Policy

The Policy entity is the active insurance contract record created after a PolicyRequest is approved and payment is confirmed. It is the central record against which all claims are filed and payments are tracked throughout the coverage period.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the issued policy.
RequestId	GUID (FK -> PolicyRequest), NOT NULL	References the originating application that produced this policy.
PolicyNumber	string, UNIQUE, NOT NULL	Human-readable policy reference number, auto-generated on issuance.
CustomerId	GUID (FK -> User), NOT NULL	References the policyholder.
AssignedAgentId	GUID (FK -> User), NOT NULL	References the Agent who processed and activated this policy.
PolicyTypeId	GUID (FK -> PolicyType), NOT NULL	References the product type governing coverage terms and amounts.
StartDate	DateTime, NOT NULL	The date from which insurance coverage becomes effective.
EndDate	DateTime, NOT NULL	The coverage expiry date, calculated as StartDate plus DurationMonths.
FinalPremium	decimal, NOT NULL	The actual annual premium amount paid, as calculated by the underwriting engine.
CoverageAmount	decimal, NOT NULL	Total maximum payout available across all claims filed against this policy.
AgentCommissionAmount	decimal, NOT NULL	Commission amount credited to the assigned agent upon successful policy activation.
Status	Enum (PolicyStatus), NOT NULL	Lifecycle state: Draft, Active, Expired, or Cancelled.
IsPaid	bool, NOT NULL	Confirms the premium has been received and the policy is financially active.
CreatedAt	DateTime, NOT NULL	Timestamp of policy record creation.

Relationships: One Policy belongs to one Customer. One Policy is linked to one PolicyRequest and one PolicyType. One Policy has many InsuranceClaims. One Policy has many Payments. One Policy has many Nominees.

9.6. Entity: InsuranceClaim

The InsuranceClaim entity records every claim filed by a customer against an active policy. It tracks the complete lifecycle of the claim from initial submission through officer investigation to final settlement or rejection.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the claim.
PolicyId	GUID (FK -> Policy), NOT NULL	References the active policy against which the claim is filed.
CustomerId	GUID (FK -> User), NOT NULL	References the Customer who filed the claim.
OfficerId	GUID (FK -> User), nullable	References the Claims Officer assigned to review this claim.
CoverageCategory	Enum (CoverageCategory), NOT NULL	The type of benefit being claimed (e.g. AccidentalDeath, PermanentPartialDisability).
IncidentDate	DateTime, NOT NULL	The date the accident or qualifying event occurred.
Description	string, NOT NULL	Detailed narrative description of the incident provided by the customer.
ClaimAmount	decimal, NOT NULL	The amount of compensation requested by the customer.
ApprovedAmount	decimal, nullable	The actual payout amount approved by the Claims Officer after assessment.
DocumentFilePath	string, nullable	Server file path to uploaded supporting evidence such as accident reports or medical certificates.
Status	Enum (ClaimStatus), NOT NULL	Current claim state: Submitted, UnderReview, Approved, Rejected, or Settled.

Relationships: Many InsuranceClaims belong to one Policy. One InsuranceClaim belongs to one Customer. One InsuranceClaim is reviewed by one ClaimsOfficer. One InsuranceClaim produces at most one Settlement record upon approval. One InsuranceClaim can have many ClaimReview records forming an audit trail.

9.7. Entity: ClaimReview

The ClaimReview entity records the detailed assessment performed by a Claims Officer when evaluating a claim. It serves as an immutable audit trail of the officer's findings and supports the final approval or rejection decision.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the review record.
ClaimId	GUID (FK -> InsuranceClaim), NOT NULL	References the claim being reviewed.
OfficerId	GUID (FK -> User), NOT NULL	References the Claims Officer conducting the review.
DisabilityPercentage	decimal, nullable	Assessed percentage of disability relevant for disability coverage categories.
RecoveryWeeks	int, nullable	Estimated weeks of recovery relevant for temporary disability claims.
FraudRiskScore	decimal, nullable	Officer's fraud likelihood score based on evidence and incident plausibility (0 to 1 scale).
Notes	string, nullable	Detailed review notes, findings, and justification for the approval or rejection decision.

Relationships: Many ClaimReviews belong to one InsuranceClaim. Each ClaimReview is authored by one ClaimsOfficer (User). Multiple reviews per claim are allowed to support multi-stage investigation.

9.8. Entity: Settlement

The Settlement entity records the final confirmed financial payout for an approved insurance claim. It is created when a Claims Officer approves a claim and initiates the settlement process, serving as the financial closure record.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the settlement record.
ClaimId	GUID (FK -> InsuranceClaim), NOT NULL, UNIQUE	References the approved claim. One-to-one relationship enforced by the unique constraint.
SettlementAmount	decimal, NOT NULL	The final monetary amount disbursed to the claimant as confirmed by the Claims Officer.

SettlementDate	DateTime, NOT NULL	Timestamp when the settlement was processed and confirmed.

Relationships: One Settlement belongs to exactly one InsuranceClaim (enforced by unique FK). The InsuranceClaim status is updated to Settled upon successful creation of this record.

9.9. Entity: Payment

The Payment entity records all financial transactions related to policy premium payments. Multiple payment records may exist per policy to accommodate failed payment retries or installment arrangements.

Field	Type / Constraint	Description
Id	GUID (PK)	Unique identifier for the payment transaction.
PolicyId	GUID (FK -> Policy), NOT NULL	References the policy this payment transaction is associated with.
Amount	decimal, NOT NULL	The monetary amount of this payment transaction.
PaymentStatus	Enum (PaymentStatus), NOT NULL	Transaction result: Pending, Paid, or Failed.
PaymentMethod	string, NOT NULL	The payment method used (e.g. Credit Card, UPI, Net Banking).
PaymentDate	DateTime, NOT NULL	Timestamp of the payment transaction.

Relationships: Many Payments belong to one Policy. A successful Paid payment triggers the Policy Status transition from Draft to Active.

9.10. Entity: Nominee

The Nominee entity records the beneficiary details designated by the policyholder for each active policy. Nominees receive the death benefit payout in the event of an accidental death claim being approved.

Field	Type / Constraint	Description

Id	GUID (PK)	Unique identifier for the nominee record.
PolicyId	GUID (FK -> Policy), NOT NULL	References the policy this nominee is associated with.
Name	string, NOT NULL	Full name of the nominated beneficiary.
Relationship	string, NOT NULL	The nominee's relationship to the policyholder (e.g. Spouse, Parent, Child).
DateOfBirth	DateTime, NOT NULL	Date of birth of the nominee.
Phone	string, NOT NULL	Contact phone number of the nominee.

Relationships: Many Nominees belong to one Policy. A policyholder can designate multiple nominees per policy for different coverage categories.

9.11. Entity Relationship Summary

Entity	Relationship	Related Entity	Cardinality
User (Customer)	submits	PolicyRequest	1 to Many
User (Agent)	evaluates	PolicyRequest	1 to Many
User (ClaimsOfficer)	reviews	InsuranceClaim	1 to Many
PolicyType	defines terms for	PolicyRequest	1 to Many
PolicyType	has coverage breakdown in	PolicyCoverage	1 to Many
PolicyRequest	results in	Policy	1 to 1
Policy	is subject of	InsuranceClaim	1 to Many
Policy	has payment records in	Payment	1 to Many
Policy	has beneficiaries in	Nominee	1 to Many
InsuranceClaim	has review records in	ClaimReview	1 to Many
InsuranceClaim	produces on approval	Settlement	1 to 1

11. Database Enumerations

All status fields in the system use strongly typed enumerations stored as integers in the database. This ensures referential integrity, prevents invalid state values, and enables the state-machine logic enforced by the service layer.

Enumeration	Values	Used In
UserRole	Admin, Agent, Customer, ClaimsOfficer	User.Role — controls dashboard access and API endpoint authorization
PolicyStatus	Draft, Active, Expired, Cancelled	Policy.Status — tracks the policy lifecycle from creation through expiry
RequestStatus	New, Assigned, UnderReview, Approved, Rejected, Accepted	PolicyRequest.Status — tracks the underwriting application pipeline
ClaimStatus	Submitted, UnderReview, Approved, Rejected, Settled	InsuranceClaim.Status — tracks the full claims processing pipeline
PaymentStatus	Pending, Paid, Failed	Payment.PaymentStatus — tracks individual premium payment transaction outcomes
CoverageCategory	AccidentalDeath, PermanentTotalDisability, PermanentPartialDisability, TemporaryTotalDisability	PolicyCoverage.CoverageCategory and InsuranceClaim.CoverageCategory

12. API Documentation Overview

The ASP.NET Core API provides highly structured, REST-compliant endpoints secured by JWT authentication with RBAC enforcement. All endpoints require a valid JWT token in the Authorization header except the Authentication endpoints.

11.1. Authentication Endpoints (/api/Auth)

Method	Endpoint	Description
POST	/register	Register a new customer account with hashed password storage.
POST	/login	Authenticate user credentials and receive a signed JWT token with role claims.

11.2. Customer Operations (/api/Customer)

Method	Endpoint	Description
POST	/policy-request	Submit a new insurance policy application with personal and medical details.
GET	/policy-requests	View status of own policy applications and underwriting results.
GET	/policies	List all active insurance policies owned by the customer.
POST	/claim	File a new insurance claim with incident details and selected coverage category.
POST	/payment	Record a premium payment to activate a pending policy.
POST	/confirm-purchase	Finalize policy purchase after underwriting approval.
GET	/notifications	Retrieve user-specific alerts, reminders, and status update messages.

11.3. Agent Operations (/api/Agent)

Method	Endpoint	Description
GET	/assigned-requests	List policy requests assigned to this agent for underwriting evaluation.
PUT	/underwriting	Submit the computed risk score and calculated premium for an assigned request.
GET	/commission-summary	View total commissions earned from successfully activated policies.
POST	/create-policy-direct	Initiate a policy application directly on behalf of a customer.

11.4. Claims Officer Operations (/api/ClaimsOfficer)

Method	Endpoint	Description
GET	/claims	List all claims assigned to this officer for review.
POST	/review	Submit a detailed assessment and decision (Approve or Reject) on a claim.
POST	/settlement	Process the final settlement payout for an approved claim.

11.5. Administrative Operations (/api/Admin)

Method	Endpoint	Description
POST	/create-agent	Provision a new Agent account with role assignments.
POST	/assign-agent-by-workload/{id}	Automatically assign a policy request to the least busy agent.
GET	/analytics	Retrieve system-wide performance metrics and business analytics.
GET	/revenue-report	Detailed financial breakdown of premiums collected and claims settled.

11.6. Support Services

Method	Endpoint	Controller	Description
POST	/upload	Document	Upload claim supporting documents or identity proof for verification.
POST	/ask	ChatBot	Submit a query to the Microsoft Semantic Kernel-powered AI chatbot.

13. Frontend Component Architecture

The frontend is built using Angular 21 with Standalone Components following a modular, role-based architecture. NG-ZORRO (Ant Design) provides the primary UI component library. State is managed using Angular Signals for component-level reactivity and RxJS Observables for all asynchronous data streams and service communication.

12.1. Core Services and State Management

Service	Technology	Responsibility
Auth	Angular Signals + RxJS	Manages JWT storage, decodes role claims, handles session lifecycle, and exposes a reactive currentUser signal consumed across all dashboards.
PolicyRequest	RxJS Observables	Handles all HTTP communication for policy applications and underwriting updates using switchMap and catchError operators.
Claim	RxJS BehaviorSubject	Manages the claim lifecycle from creation to settlement, using BehaviorSubject to track current claim state across components.
Notification	RxJS interval + HttpClient	Polls the notifications endpoint at regular intervals using RxJS interval and mergeMap, delivering real-time alerts to the NotificationBell component.
File	RxJS + HttpClient	Handles multipart document uploads using RxJS for progress tracking and error recovery.

12.2. Public Components

Component	Description
Login	Secure credential entry with JWT-based authentication, Angular Reactive Forms validation, and role-based redirect on success.
Register	New customer onboarding form with comprehensive validation for personal details.
PublicPage	Landing page explaining available insurance products and platform benefits.

12.3. Customer Dashboard Components

Component	Description
PolicyMarketplace	Interactive card-based display of available insurance products with coverage details and premium estimates.
PolicyApplication	Multi-step form wizard for submitting policy requests including medical history and document upload.
MyPolicies	Centralized view of active policies, coverage details, nominee information, and payment history.
ClaimWizard	Guided step-by-step form for reporting accidents, selecting coverage category, and uploading evidence.
NomineeManagement	Interface to add, edit, or remove nominees for each active policy.

12.4. Agent Dashboard Components

Component	Description
UnderwritingQueue	Prioritized list of policy requests assigned to this agent for risk evaluation.
RiskAssessment	Interface for entering risk evaluation parameters and viewing the system-calculated risk score and premium.
CommissionTracker	Dashboard summarizing commissions earned from activated policies.

12.5. Claims Officer Dashboard Components

Component	Description
ClaimReviewQueue	Tabular list of assigned claims with status indicators and claim amounts.
ClaimDetail	Full detail view including incident description, uploaded documents, policy coverage summary, and review history.
SettlementProcessor	Form for entering the final approved settlement amount and submitting the settlement decision.

12.6. Admin Console Components

Component	Description
UserManagement	CRUD interface for Agent and Claims Officer accounts with workload display.
SystemOverview	Global summary of all active policies, pending requests, and open claims.
AnalyticsDashboard	Visual dashboards for revenue, claim metrics, and business performance.
RevenueReport	Detailed financial reports with date range filters for premiums and claims.

12.7. Shared and Layout Components

Component	Description
AppHeader	Dynamic navigation bar adapting its menu items based on the decoded JWT role claim.
AppSideNav	Role-specific sidebar for dashboard sub-navigation with active route highlighting.
ChatBot	Floating AI assistant powered by Microsoft Semantic Kernel, available across all dashboards.
Public Page	Landing page consisting of testimonials,tour component,advertisment

14. Key Features and Technical Highlights

13.1. Key Features

- **Stateless JWT Authentication and RBAC:** A highly secure, stateless API architecture where the frontend dynamically configures navigation based on JWT role claims and every backend endpoint enforces [Authorize(Roles=...)] protection.
- **Intelligent Underwriting Engine:** A multiplier-based risk model evaluates Age, Occupation, Personal Habits, and Medical History. Applications with a composite risk score greater than 5 are automatically rejected, ensuring consistent and unbiased eligibility decisions.
- **Least Workload Assignment Algorithm:** Automated, equitable distribution of policy requests to agents and claims to officers based on real-time active workload metrics, eliminating manual assignment overhead.
- **Microsoft Semantic Kernel AI ChatBot:** An AI orchestration layer built on Microsoft Semantic Kernel provides an intelligent assistant that understands natural language queries, explains coverage categories, and guides customers through the claims process.
- **Dual Reactive State Architecture:** Angular Signals provide fine-grained synchronous component reactivity for UI state, while RxJS manages all asynchronous concerns including HTTP streams, polling, and complex event transformation pipelines.
- **Multi-Stage Claim Processing Pipeline:** Declarative state-machine logic enforces valid claim status transitions. Document verification, officer investigation, and settlement processing are mandatory sequential steps that cannot be bypassed.
- **Configurable Coverage Category System:** Each policy type defines multiple coverage categories with independently configured payout percentages and compensation limits.

13.2. Development Challenges and Solutions

Challenge	Solution Implemented
Multi-Role Access Control	Implemented strict RBAC with JWT role claims and [Authorize(Roles=...)] on every endpoint. Angular RoleGuard prevents unauthorized route access. Angular Signals expose a reactive currentUser to all dashboard components.
Dynamic Underwriting Engine	Abstracted risk evaluation into independent evaluator classes following the Strategy pattern, making the engine easily extensible with new risk dimensions without modifying existing code.

Challenge	Solution Implemented
Workload Balancing Without Race Conditions	Assignment logic is wrapped in a database transaction with optimistic concurrency control to prevent duplicate assignments when multiple requests arrive simultaneously.
Document Upload and Storage	Multipart file uploads with server-side validation store files in wwwroot folder in backend and persist only file paths in the database to avoid BLOB performance degradation.
Complex Asynchronous Frontend Flows	RxJS operators (switchMap, combineLatest, debounceTime) were used to compose complex asynchronous workflows including dependent API calls, real-time polling, and concurrent multi-file uploads predictably.

15. Future Enhancements

The architecture of this system is designed for extensibility, allowing seamless integration of advanced enterprise features in subsequent phases:

- **Real Payment Gateway Integration:** Replace simulated financial transactions by integrating Razorpay, Stripe, or PayPal webhooks for real-time payment processing with automated reconciliation.
- **AI-Based Fraud Detection:** Extend the Microsoft Semantic Kernel integration with ML plugins trained on historical claim data to automatically score incoming claims for fraud likelihood.
- **Email and SMS Notification Service:** Integrate production-grade notification providers (SendGrid for email, Twilio for SMS) using background worker queues for reliable asynchronous lifecycle event delivery.
- **Mobile Application:** Develop a cross-platform mobile app (React Native or Flutter) with on-the-go policy management, claim filing with photo capture, and push notifications.
- **Advanced Analytics and Business Intelligence:** Integrate a BI module with predictive analytics for claim forecasting, agent performance scoring, and portfolio risk modeling.
- **Document OCR and Verification:** Implement optical character recognition to automatically extract and validate information from uploaded identity and medical documents.
- **Policy Renewal Automation:** Build automated renewal workflows with configurable reminder schedules and pre-filled renewal forms based on existing policy data.

16. Conclusion

The Personal Accident Insurance Management System successfully demonstrates the principles of modern software engineering applied to complex enterprise business logic in the insurance domain. By systematically replacing manual operations with secure, scalable, and rigidly role-driven digital workflows, this system represents a highly viable solution for optimizing the personal accident insurance sector.

The platform leverages a Clean Architecture backend built on ASP.NET Core 8.0 with Entity Framework Core, AutoMapper, and Microsoft Semantic Kernel for AI capabilities, combined with a modern Angular 21 frontend utilizing NG-ZORRO components, Angular Signals, RxJS, Driver.js, and Lucide Icons to deliver a seamless and reactive experience across all four user roles. The full source code is available at https://github.com/abshek7/ABHISHEK.

Through meticulous database design, strict RBAC implementation, and comprehensive business logic orchestration including the multiplier-based Underwriting Engine with an automatic rejection threshold of 5, the Least Workload Assignment Algorithm, and the multi-stage Claim Processing Pipeline, the system delivers significant measurable value to both insurance providers and policyholders, positioning this platform as a modern, future-ready solution for digital accident insurance operations.